Report for Laboratory 5: Introduction to Interrupts

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

June 21, 2026

Abstract. This lab explored the use of interrupts and how interrupts are handled by the FPGA of the myRIO-1900. The interrupts will respond to an external signal, which is the trigger from an SPDT (Single Pole, Double Throw) switch. This lab also examined the mechanical properties of the SPDT switch with respect to the electrical signals it affects and how the use of a flip-flop to store the state of a switch will help smooth the transition and prevent noise from affecting the system. The flip-flop circuit used in this lab was a NAND debouncing circuit.

1 Introduction

We connected an SPDT switch to the myRIO DIO0 on Connector A, as shown in Figure 1. Connector A on the myRIO is pin 11. Other pins that were identified and used in this lab include:

- DIO0 (pin 11)
- GND (pin 12)
- +3.3V(pin 33)

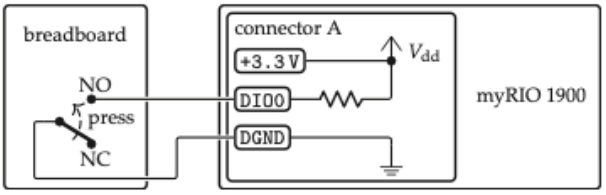

Figure 1: Connecting SPDT directly to myRIO DIO0. Picone et al. (2024).

The objectives of this lab were:

1. To implement a system with an external interrupt
2. To program an interrupt service routine
3. To program multiple threads
4. To prototype a digital logic circuit
5. To debounce a mechanical switch

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Rigol DHO914S Digital Oscilloscope with 25Mhz function generator
- Rigol 712 DC power supply (to supply power to the NAND Integrated Circuit (IC)) [1]
- breadboard
- 1x NAND Integrated Circuit (IC) - SN7438N
- 4x 2.2 kΩ resistors
- Cables:
  BNC with alligator clips
  jumper cables

2.2 Using a debouncing circuit

After experimenting with the SPDT switch connected directly to the myRIO and observing the signal characteristics on the oscilloscope, we implemented a debouncing circuit using a NAND gate to smooth the transitions and prevent noise from affecting the system. The schematic of the debouncing circuit is shown in Figure 2.

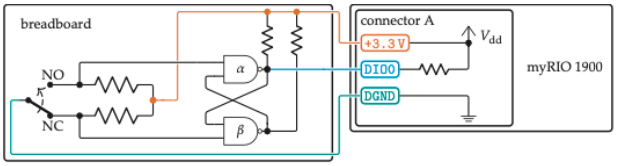

Figure 2: Schematic of the debouncing circuit using a NAND gate. Picone et al. (2024).

The actual implementation of the debouncing circuit is shown in Figure 3. The NAND IC was powered by the Rigol 712 DC power supply set at +3.3V with the ground connected to the myRIO. Pin 11 of the myRIO is connected to the output of the first NAND gate (pin 3).

[1] This was optional because we could have drawn power from the function generator of the oscilloscop or from the myRIO. But because we used this, we had to make sure the ground from this power supply is tied to the ground of the myRIO and the entire circuit so as to establish a common ground.

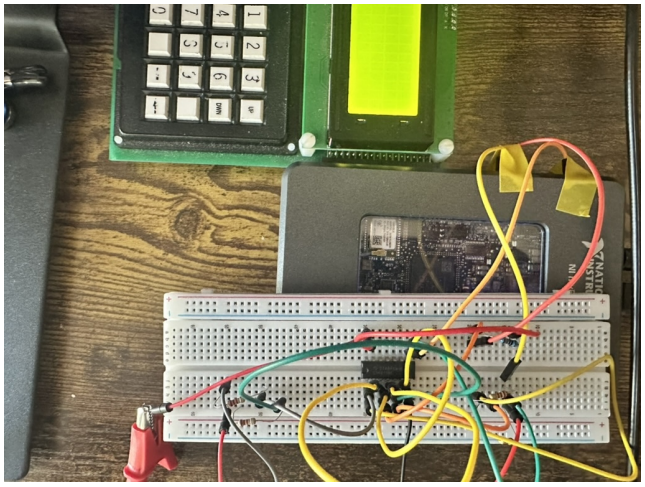

Figure 3: Actual implementation of the debouncing circuit. Picone et al. (2024).

3 Laboratory Exploration and Evaluation

The complete source code for the lab is available on GitHub at `https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab5/main.c`.

A video demonstration of the lab is avaiable at `https://github.com/juliansoh/ME477/blob/main/Reports/Lab5/videos/IMG_0052.MOV`.

3.1 Problem L4.1

Determine the exact number of clock cycles for the wait() function of listing 4.2 to execute, accounting for all instructions (use table 4.6). From that, calculate the delay interval in milliseconds. A spreadsheet is convenient for making this calculation.

The mapping of the wait() function to assembly code is shown in Figure 4. The loop in the wait() function is highlighted in the dotted box and is the part of the code that executes 417000 times.

The setup overhead of the wait() function only runs once, as does the loop cleanup after i == 0.

```
WAIT:
  PUSH {R11}
  ADD R11, SP, #0
  SUB SP, SP, #12
  MOV R3, #417000
  STR R3, [R11, #-8]
  B START
LOOP: LDR R3, [R11, #-8]
  SUB R3, R3, #1
  STR R3, [R11, #-8]
START: LDR R3, [R11, #-8]
  CMP R3, #0
  BNE LOOP
  NOP
  ADD SP, R11, #0
  LDMFD SP!, {R11}
  BX LR
```

Overhead / setup

```
i = 417000;
while (i > 0) {
  i--;
}
```

End loop execution when i == 0

Figure 4: Mapping the wait() to assembly code.

Function overhead/setup

Instruction Type	Mnemonic	Clock Cycles
Load/store registers	LDR, STR	2
Branch	BNE, B, BX	0
Stack	PUSH, LDMFD	2
Arithmetic	ADD, SUB, CMP	1
Move	MOV	1
No operation	NOP	1

Table 1: Opcode execution times (CPU cycles) for instructions used in `wait()`

$$\begin{aligned} \text{PUSH} &: 2 \\ \text{ADD} &: 1 \\ \text{SUB} &: 1 \\ \text{MOV} &: 1 \\ \text{STR} &: 2 \\ \text{B} &: 0 \end{aligned}$$

$$\text{Total setup cycles} = 2+1+1+1+2 = 7$$

Loop interation Each iteration of the loop executes the following instructions:

$$\begin{aligned} \text{LDR} &: 2 \text{ cycles} \\ \text{SUB} &: 1 \text{ cycle} \\ \text{STR} &: 2 \text{ cycles} \\ \text{LDR} &: 2 \text{ cycles} \\ \text{CMP} &: 1 \text{ cycle} \\ \text{BNE} &: 0 \text{ cycles} \end{aligned}$$

Total cycles per loop iteration:

$$2+1+2+2+1+0 = 8 \text{ cycles}$$

The loop executes 417000 times, so total loop cycles:

$$417000 \times 8 = 3,336,000 \text{ cycles}$$

Exit instructions:

$$\begin{aligned} \text{NOP} &: 1 \\ \text{ADD} &: 1 \\ \text{LDMFD} &: 2 \\ \text{BX} &: 0 \end{aligned}$$

$$\text{Total exit cycles} = 1+1+2 = 4$$

Therefore, the total clock cycles for wait() is:

$$\text{Total cycles} = 3,336,000+7+4 = 3,336,011 \text{ cycles}$$

Time Calculation The processor clock frequency is:

$$f = 667 \text{ MHz}$$

Clock period:

$$T = \frac{1}{667 \times 10^6}$$

Total execution time:

$$t = \frac{3,336,011}{667 \times 10^6}$$

$$t \approx 0.00500 \text{ seconds} = 5.00 \text{ ms}$$

3.2 Problem L4.2

Examine the circuit board attached to connector C of the myRIO. The normally open push-button switches, when closed, connect DIO channels 1 and 5 to Vcc = 5 V when pressed, as shown in figure 4.19. Note: These channels have pull-down resistors. Use the oscilloscope to view the waveform produced by your program. For example, use N= 5, M= 3.

From the oscilloscope, we observed the PMW waveform as shown in Figure 5.

Figure 5: Image of the PWM waveform from the Oscilloscope.

3.3 Problem L4.3

Compare your program's printed output (on the LCD) to the steady state motor speed as measured with an optical tachometer.

Figure 6 and Figure 7 show the readings from the LCD to be comparable to the readings taken with the optical tachometer. This is when we used the frequency measurement from the oscilloscpe. However, when we used the calculated time of wait() of approximately 5ms, the rpm reported on the LCD is about double what the ground truth value of the optical tachometer. This is because the BTI value is too small and is likely because the additional processor cycles for low() and print_lcd() were not taken into consideration. The rpm reported correctly is about 1477 rpm.

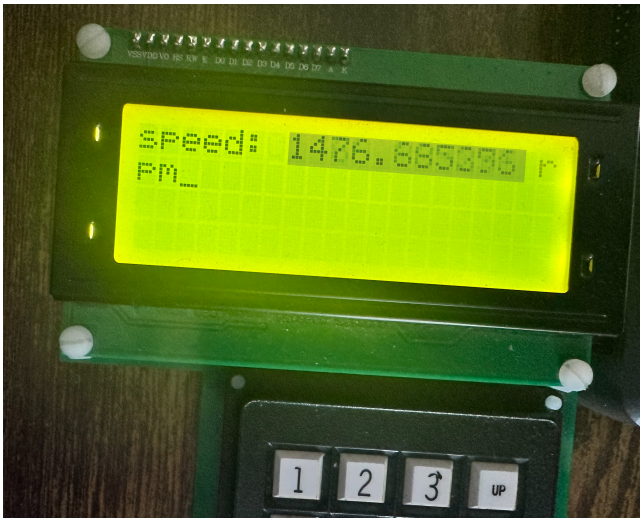

Figure 6: Image of the LCD display showing the motor speed.

3.4 Problem L4.4

Use the oscilloscope to view the PWM waveform produced by your program, and to measure the actual length of a BTI. Is it what you expect? If not, why not?

Refer to Figure 5 again and note the measurements taken on the waveform were:

- Period: 46 ms (N)
- Width (high): 30 ms (M)
- Duty cycle: 65%
- M/N = 30/46 = 0.65

We entered N=5 and M=3, so the expected duty cycle is 3/5 = 0.6 or 60%. The actual duty cycle is 65%, which is slightly higher than expected. This could be due to timing inaccuracies in the program or the oscilloscope measurements.

3.5 Problem L4.5

Repeat lab problem L4.4 while printing the speed (pressing the switch). What does the oscilloscope show has happened to the length of the BTI? What's going on!?

When the print button is pressed, the oscilloscope shows that the length of the BTI (Base Time Interval) has changed. The new measured values were:

- Period: 56 ms (N)
- Width (high): 46 ms (M)
- Duty cycle: 81%
- M/N = 46/56 = 0.82

3.6 Problem L4.6

Describe how you made the measurements of lab problems L4.4 and L4.5 and discuss any limitations in the accuracy of

Figure 7: Image of the optical tachometer measurement.

the timing of the output. In lab exercise 6, we will find ways of overcoming this limitation.

Refer to Figure 5 that shows the screen of the oscilloscope. The measurements taken are shown to the right of the waveform: Period, +Width, and +Duty.

These measurements were taken by using the oscilloscope's measurement tools and selecting these parameters from the oscilloscope's menu.

The limitations of using these tools is that the measurements are taken in real time and may be subject to noise and other sources of error. We could just take the average of the measurements to improve accuracy. Nevertheless, this approach is more accurate than manual measurements or visual inspection.

3.7 Problem L4.7

After you have your code running as described above, try this: Record the velocity step response of the DC motor, save it to a file, and plot it in MATLAB. Here's how:

Please see separate .mat file in the repo.

3.8 Problem L4.8

You may have noticed that when M = 1, the FSM does not function as desired. Address the following issues:

1. What is wrong?
2. How would modifying the state transition diagram correct this problem?
3. How would you modify the state transition table?
4. Modify your program to correct the M = 1 case. Test the result.

When M=1, the motor starts spinning but when the print button is pressed, the motor stops. This is because the FSM is not properly handling the case where M=1. Pressing the Stop button does not terminate the program either. Looking at High(), we notice that condition clock == 1 is never met when M =1 because of clock++. Therefore, changing this condition to M>=1 fixed the problem.

3.9 Problem L4.9

A programmer has written following code in an attempt to initialize the encoder.

```
MyRio_Encoder *encC0;
EncoderC_initialize(myrio_session, encC0);
```

It's incorrect. What issues could this code cause, and why?

The declaration initializes a pointer to the `MyRio_Encoder` structure but does not allocate memory for the structure itself. Therefore, when the code attempts to read or write to this memory location, it will access invalid or nonexistent memory.

Instead, the `MyRio_Encoder` structure should be declared. The address operator can then be used to pass a pointer to the `MyRio_Encoder` structure. See page 230 of Picone et al. (2024).

4 Author Contributions

There is only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.